

Student Answer Sheet

Observability & Centralized Logging (Chapter 5)

Course: Systems Reliability Engineering (SRE)
Lab: Observability & Centralized Logging

Group Number: 5 - 4

Students:

Madaoui Sara
Madoui Roufaida
Belkerrouche Asma
Charif Lydia
Zaoudi Hadil
Bokreta Hadjer

Contents

1	The Challenge (Section 2)	3
2	Part 1: The Three Pillars (Section 3)	4
2.1	Pillar Characteristics Table	4
2.2	Diagnostic Question (Section 3.1)	4
3	Part 2: ELK Technical Analysis (Section 4)	5
3.1	Beats & Elastic Agent	5
3.2	Logstash & Grok	5
3.3	Elasticsearch & The Inverted Index	5
3.4	Kibana	6
4	Part 3: WEF Design Challenge (Section 5)	6
4.1	Architecture Evaluation Table	6
4.2	Windows Server 2025 Innovation (Section 5.2.1)	7
4.3	Forensic Prioritization (Section 5.2.2)	7
5	Part 4: SLIs and Error Budgets (Section 6)	7
5.1	Error Budget Calculation (Section 6.1)	7
5.2	SLI Proposal (Section 6.2)	8
5.3	The “SRE Clamp” (Section 6.3)	8
6	Part 5: Practical Implementation (Section 7)	8
6.1	Task 5.1: docker-compose.yml	8
6.2	Task 5.2: filebeat.yml (Nginx Server)	10
6.3	Task 5.2: filebeat.yml (Microservice Server)	11
6.4	Task 5.3: winlogbeat.yml	11
6.5	Task 5.4: logstash.conf	12
6.6	Task 5.5: Visual Proof	15
7	Final Deliverable: System Interaction Map (Section 8)	18
7.1	Data Source 1: Ubuntu Linux Web Server (Nginx)	18
7.2	Data Source 2: Windows Server Virtual Machine (Security Event Log)	19
7.3	Data Source 3: Checkout Microservice (Containerised Python/Flask Application)	20

1 The Challenge (Section 2)

Question

Explain why binary “up/down” monitoring failed in this instance. Why is monitoring “pre-defined questions” insufficient for identifying p99 latency spikes in a distributed microservices environment?

Answer

Binary “up/down” monitoring has a basic problem: it can only check simple yes/no questions like “Is port 443 open?” or “Did the server return HTTP 200?”. These checks tell us if the service is running, but they don’t tell us anything about how fast it is or if users are happy with it.

When a checkout request goes through multiple services (authentication, inventory, payment, shipping), each service might return HTTP 200 and seem fine on its own. But if all of them are a bit slow, the total time adds up and users experience bad p99 latency. The problem is that traditional monitoring was never set up to check for this.

p99 latency is an “unknown-unknown”—a problem nobody expected, so it wasn’t being monitored. To fix this, we need a system that collects detailed logs, metrics, and traces so we can ask questions about the data after a problem happens, without having to predict every possible issue beforehand.

2 Part 1: The Three Pillars (Section 3)

2.1 Pillar Characteristics Table

Pillar Name	Data Characteristic	Specific Use Case (E-Commerce)
Logs	Discrete events, high cardinality, unstructured or semistructured text. Each log is a timestamped record of one specific event.	Recording each checkout attempt: <code>user id=5821 action=payment submit status=error timestamp=....</code> Used to reconstruct exactly what happened during a failed transaction.
Metrics	Aggregated data points, lower cardinality, time-series. Each metric is a snapshot of system state at a point in time.	Tracking requests-per-second, CPU%, memory usage, and critically p99 checkout response time over time. Metrics would show the latency spike on a graph.
Traces	Distributed request lifecycle tracking. One trace = one user request spanning multiple services. Contains “spans” for each service hop.	A single checkout trace showing: Auth=12 ms, Inventory=8 ms, Payment=2,400 ms, Shipping=9 ms. Immediately reveals the Payment service as the bottleneck even though it returned HTTP 200.

Table 1: Three Pillars of Observability

2.2 Diagnostic Question (Section 3.1)

Question

How do Traces specifically resolve the problem of identifying bottlenecks in a distributed request lifecycle where every individual service appears healthy but the overall response is delayed?

Answer

Traces solve the bottleneck problem by following a single request through all the services it touches. Each request gets a unique ID, and every service records how long it took to do its job. When we look at all these times together, we can see exactly where the request spent most of its time.

In our checkout example, all services returned HTTP 200 (success), so we might think everything is working fine. But when we look at the trace, we see the real story:

authentication took 12 ms, inventory took 8 ms, but payment took 2,400 ms. This is why the checkout was slow, even though all the services said they succeeded. Traces answer the question “where is the time going?” instead of just “did it work?” — which is what monitoring tools cannot do.

3 Part 2: ELK Technical Analysis (Section 4)

3.1 Beats & Elastic Agent

Answer

Beats are small programs that run on servers and collect data like logs and metrics. They don't use much computer power, which is important for servers that are already busy. They send the data they collect to Logstash for processing. For example, Filebeat watches log files and sends new log lines to Logstash as they appear.

When you have many servers, managing each one's settings separately becomes very difficult and error-prone. Elastic Agent fixes this problem by being one agent that collects logs, metrics, and security data all at once. Instead of changing settings on each server manually, you can use Kibana to change the settings for all agents at the same time. This makes managing the system much easier.

3.2 Logstash & Grok

Answer

Logstash takes messy, unstructured log text and converts it into organized data that we can use. It uses patterns called Grok filters to break down log lines into structured fields. For example, a raw Nginx log line looks like a jumbled string of text, but Grok converts it into separate fields like client IP, HTTP method, request path, status code, and response size.

Logstash also adds extra information to the logs before saving them. For example, it can look up where a client's IP address is located (city, country, etc.) or identify what device is being used. This happens before the data is stored, so every time we search the logs later, all this extra information is already there. This helps us quickly see patterns like “all the payment errors are from users in a specific country” or “all the errors are from mobile devices”.

3.3 Elasticsearch & The Inverted Index

Answer

Elasticsearch uses something called an inverted index to search through data quickly. Instead of checking every log entry one by one (which would be very slow), it creates a list of all unique words and which logs contain them. So when we search for “status code 500”, it can instantly find all the logs with that code without checking every single log.

This is much better for logs than traditional databases because logs come in many different formats, have lots of data, and we usually just write them once and read them many times. Elasticsearch lets us search through huge amounts of logs (terabytes) in just

seconds, even across months of old data. This makes it practical to investigate problems by looking at historical logs.

3.4 Kibana

Answer

Kibana is the interface where we can look at all the data collected by the ELK stack. The best thing about it is that we can jump between logs, metrics, and traces easily. For example, if we see that checkout latency was bad during a specific 3-minute window, we can apply a time filter and Kibana will automatically show us all the logs from that same time window. If we see a Trace ID in a log, we can click on it to see the trace view and find out exactly which service was slow.

Kibana also lets us create dashboards that show both business metrics (like how much money was made) and technical metrics (like p99 latency). This helps show non-technical people why a technical problem is important for the business.

4 Part 3: WEF Design Challenge (Section 5)

4.1 Architecture Evaluation Table

Feature	Source-Initiated (Push)	Collector-Initiated (Pull)
Scalability	Highly scalable. Each source manages its own connection. Adding new sources requires no reconfiguration of the collector.	Less scalable. Collector must maintain a list of all sources and poll each one. Adding sources requires updating the collector's subscription list.
Firewall Configuration	Simpler for sources behind NAT or strict firewalls. Sources open outbound connections only (port 5985). The collector needs one inbound port open.	Complex. The collector must be able to reach every source machine. Every source must allow inbound connections — difficult for machines behind firewalls or NAT.
Suitability for Remote Laptops	Ideal. Laptops can push logs whenever they connect to any network, including VPN. The collector doesn't need to know the laptop's current IP address.	Poorly suited. Laptops have dynamic IPs and are frequently offline or on external networks. The collector cannot reliably initiate connections to them.

Table 2: WEF Architecture Comparison

4.2 Windows Server 2025 Innovation (Section 5.2.1)

Answer

Normally, Kerberos authentication requires a direct connection to a Domain Controller (DC) to get credentials. This is a problem for remote laptops, VPNs, and field devices because they might not have that connection. When this happens, the system falls back to NTLM, which is an older, less secure authentication method that has known security problems.

Windows Server 2025 solves this with two new features:

1. **IAKerb** allows a remote device to send authentication requests through another server that does have a connection to the Domain Controller, so it doesn't need direct access.
2. **Local KDC** allows a machine to issue its own credentials without needing the Domain Controller at all.

For WEF (Windows Event Forwarding), this means remote laptops can now use Kerberos authentication instead of NTLM when connecting to the log collector, even without a VPN connection to the Domain Controller. This makes the connection more secure and prevents certain types of attacks.

4.3 Forensic Prioritization (Section 5.2.2)

Question

Identify the two specific Windows Event IDs for logon success and failure:

Event Type	Event ID
Logon Success	4624
Logon Failure	4625

5 Part 4: SLIs and Error Budgets (Section 6)

5.1 Error Budget Calculation (Section 6.1)

Given

- SLO: 99.95%
- Total minutes in 30 days: 43,200

Calculation

$$\begin{aligned}\text{Error Budget} &= \text{Total minutes} \times (1 - SLO) \\ &= 43,200 \times (1 - 0.9995) \\ &= 43,200 \times 0.0005 \\ &= 21.6 \text{ minutes per month}\end{aligned}$$

Final Answer

21.6 minutes per month

5.2 SLI Proposal (Section 6.2)**Proposed SLI**

“The proportion of checkout requests that complete with an end-to-end response time of less than 800 ms, measured at the 99th percentile (p99) over a 5-minute rolling window.”

Justification

Using p99 (99th percentile) latency as an SLI is better than using average (mean) latency. If we use average, it gets fooled by fast requests. For example, if 98% of checkouts complete in 100 ms but 2% take 5,000 ms, the average is only 198 ms, which sounds fine. But p99 shows the truth—5,000 ms—which matches what users are actually experiencing. Since customers who have slow transactions are most likely to leave without buying, and the most valuable orders might be the slow ones, p99 latency is a much better measure of whether customers are happy with the system.

5.3 The “SRE Clamp” (Section 6.3)**Answer**

If the checkout slowness uses up all 21.6 minutes of the monthly error budget, the SRE team puts a freeze on releasing new features. No new code can go live until either the month ends and the budget resets, or the performance problem is fixed.

This rule, called the SRE Clamp, makes sure that developers and SRE teams have the same goal. Feature releases use up “reliability budget”; when the budget is empty, the team has to focus on fixing problems instead of adding new features. This stops a bad situation where new features get added to a broken system, making things worse. It’s not a punishment—it’s just a rule everyone agreed to beforehand, so there’s no fighting about priorities when there’s an outage.

6 Part 5: Practical Implementation (Section 7)**6.1 Task 5.1: docker-compose.yml**

```
version: "3.8"
volumes:
  esdata01:
    driver: local
  kibana_data:
    driver: local
  logstash_data01:
    driver: local
networks:
  default:
```



```

    name: elastic
    external: false

services:
  elasticsearch:
    image:
      docker.elastic.co/elasticsearch/elasticsearch:${STACK_VERSION}
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - cluster.name=${CLUSTER_NAME}
      - "ES_JAVA_OPTS=-Xms${ES_MEM_LIMIT} -Xmx${ES_MEM_LIMIT}"
    ports:
      - "${ES_PORT}:9200"
    volumes:
      - esdata01:/usr/share/elasticsearch/data
    healthcheck:
      test: ["CMD-SHELL", "curl -s http://localhost:9200 | grep -q 'cluster_name'"]
      interval: 10s
      timeout: 10s
      retries: 120

  logstash:
    image: docker.elastic.co/logstash/logstash:${STACK_VERSION}
    container_name: logstash
    ports:
      - "5044:5044"
    volumes:
      - logstash_data01:/usr/share/logstash/data
      -
        ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf
    environment:
      - xpack.monitoring.enabled=false
    depends_on:
      elasticsearch:
        condition: service_healthy

  kibana:
    image: docker.elastic.co/kibana/kibana:${STACK_VERSION}
    container_name: kibana
    ports:
      - "${KIBANA_PORT}:5601"
    volumes:
      - kibana_data:/usr/share/kibana/data
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
      - ELASTICSEARCH_USERNAME=kibana_system
      - ELASTICSEARCH_PASSWORD=${KIBANA_PASSWORD}
      - XPACK_SECURITY_ENCRYPTIONKEY=${ENCRYPTION_KEY}

```

```

-
  XPACK_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY=${ENCRYPTION_KEY}
depends_on:
  elasticsearch:
    condition: service_healthy

checkout-microservice:
  build:
    context: ./microservice
    dockerfile: DockerFile
  container_name: checkout-microservice
  ports:
    - "5000:5000"
  networks:
    - default
  depends_on:
    elasticsearch:
      condition: service_healthy
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"

filebeat:
  image: docker.elastic.co/beats/filebeat:${STACK_VERSION}
  container_name: filebeat
  user: root
  command: filebeat -e --strict.perms=false
  volumes:
    -
      ./microservice/filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
    - /var/lib/docker/containers:/var/lib/docker/containers:ro
    - /var/run/docker.sock:/var/run/docker.sock:ro
  depends_on:
    elasticsearch:
      condition: service_healthy
    logstash:
      condition: service_started
  networks:
    - default

```

6.2 Task 5.2: filebeat.yml (Nginx Server)

```

# INPUTS SECTION
# Tells Filebeat WHAT to read
filebeat.inputs:
  - type: log
    enabled: true
    paths:

```

```
    - /var/log/nginx/access.log
    tags: ["nginx", "webserver"]

# OUTPUT SECTION
# Tells Filebeat WHERE to send the logs
output.logstash:
  hosts: ["192.168.230.1:5044"]

# LOGGING SECTION
# Tells Filebeat how to log its OWN activity
logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat
  keepfiles: 7
```

6.3 Task 5.2: filebeat.yml (Microservice Server)

```
filebeat.inputs:
  - type: container
    enabled: true
    paths:
      - /var/lib/docker/containers/*/log
    processors:
      - add_docker_metadata:
          host: "unix:///var/run/docker.sock"

processors:
  - drop_event:
      when:
        not:
          equals:
            container.name: "checkout-microservice"

output.logstash:
  hosts: ["logstash:5044"]

logging.level: info
```

6.4 Task 5.3: winlogbeat.yml

```
winlogbeat.event_logs:
  - name: Security
    event_id: 4624,4625

output.logstash:
  hosts: ["192.168.230.1:5044"]
```

```
logging.level: info
```

6.5 Task 5.4: logstash.conf

```
input {
  beats {
    port => 5044
  }
}

filter {

  # NGINX LOGS from Ubuntu VM Filebeat
  if "nginx" in [tags] or "nginx" in [source] or "access" in
    [log][file][path] {

    grok {
      match => { "message" =>
        "(?:%{IP:client_ip}|%{IPV6:client_ip})\_%{DATA:ident}\_%{DATA:auth}\[%{HTTPDATE:timestamp}]\_%{WORD:http_method}\_%{DATA:request_path}\_HTTP/%{NUMBER:http_version}\_%{NUMBER:status_code:int}\_%{NUMBER:response_size:int}\_%{DATA:referrer}\_%{DATA:user_agent}\" }
    }

    date {
      match => [ "timestamp", "dd/MMM/yyyy:HH:mm:ss_Z" ]
      target => "@timestamp"
    }

    mutate {
      convert => {
        "status_code" => "integer"
        "response_size" => "integer"
      }
      add_field => {
        "environment" => "lab"
        "server" => "nginx"
        "service_type" => "nginx"
      }
    }

    if [status_code] >= 400 {
      mutate { add_tag => [ "error" ] }
    }

    if [status_code] == 200 {
      mutate { add_tag => [ "success" ] }
    }
  }
}
```

```
}

# MICROSERVICE LOGS from Docker container Filebeat
else if [container][name] == "checkout-microservice" {

    json {
        source => "message"
        target => "app"
    }

    mutate {
        add_field => {
            "service_type" => "microservice"
            "environment" => "lab"
        }
        add_tag => [ "microservice", "checkout" ]
    }

    if [app][level] == "ERROR" {
        mutate { add_tag => [ "error" ] }
    }

    if [app][level] == "WARNING" {
        mutate { add_tag => [ "warning" ] }
    }

    if [app][level] == "INFO" {
        mutate { add_tag => [ "success" ] }
    }
}

# WINDOWS LOGS from Winlogbeat
else {
    mutate {
        add_field => {
            "service_type" => "windows"
            "environment" => "lab"
        }
    }
}

}

output {

    # NGINX nginx index
    if [service_type] == "nginx" {
        elasticsearch {
            hosts => [ "http://elasticsearch:9200" ]
            index => "nginx-logs-%{+YYYY.MM.dd}"
        }
    }
}
```

```
}

# MICROSERVICE microservice index
else if [service_type] == "microservice" {
  elasticsearch {
    hosts => [ "http://elasticsearch:9200" ]
    index => "microservice-logs-%{+YYYY.MM.dd}"
  }
}

# WINDOWS windows index
else {
  elasticsearch {
    hosts => [ "http://elasticsearch:9200" ]
    index => "windows-logs-%{+YYYY.MM.dd}"
  }
}

stdout {
  codec => rubydebug
}
}
```

6.6 Task 5.5: Visual Proof

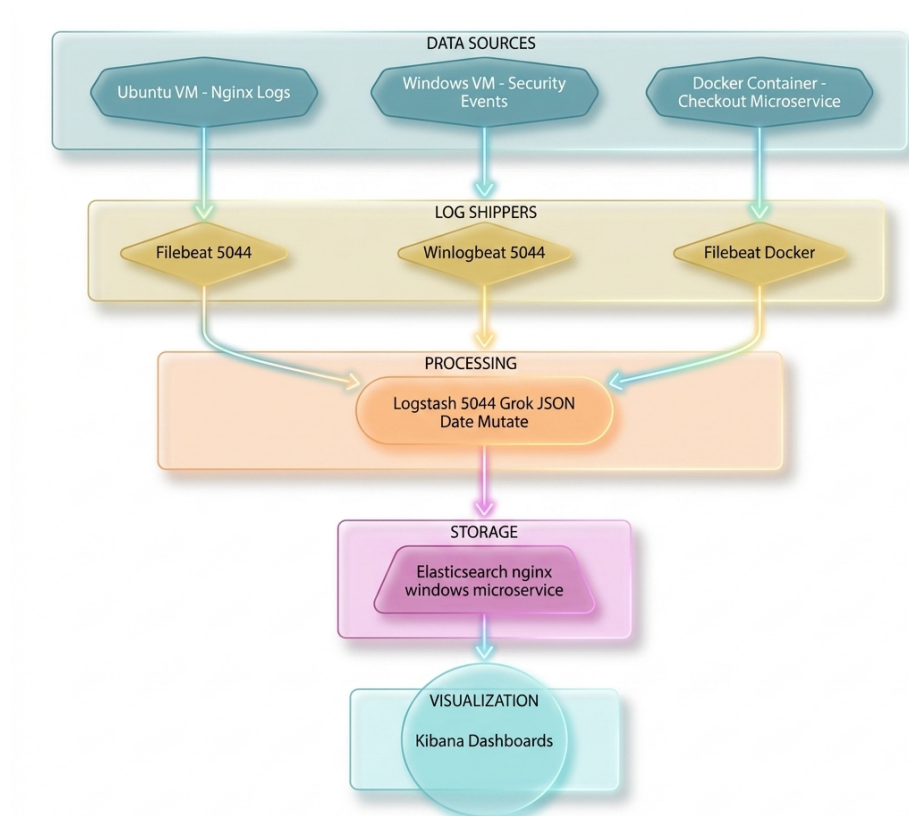


Figure 1: Architecture diagram

Figure 1: Architecture diagram

```

PS C:\Users\LENOVO User\Downloads\ELK\ELK-Stack> docker exec elasticsearch curl -s "http://localhost:9200/_cat/indices?v"
health status index      uuid                                pri rep docs.count docs.deleted store.size
ze pri.store.size dataset.size
yellow open  windows-logs-2026.05.25  8jDLBHFkSoSaZpc68pDSYw  1  1    151570         0    68.7
mb      68.7mb      68.7mb
yellow open  nginx-logs-2026.05.25    pbnsz1LMTIGzybqaGBi6Qw  1  1      300         0    303.9
kb      303.9kb     303.9kb
yellow open  microservice-logs-2026.05.25 m1R-fQJkT_yJdfaR1Q8exg  1  1      458         0    703.4
kb      703.4kb     703.4kb
PS C:\Users\LENOVO User\Downloads\ELK\ELK-Stack>
  
```

Figure 2: elasticsearch indexes

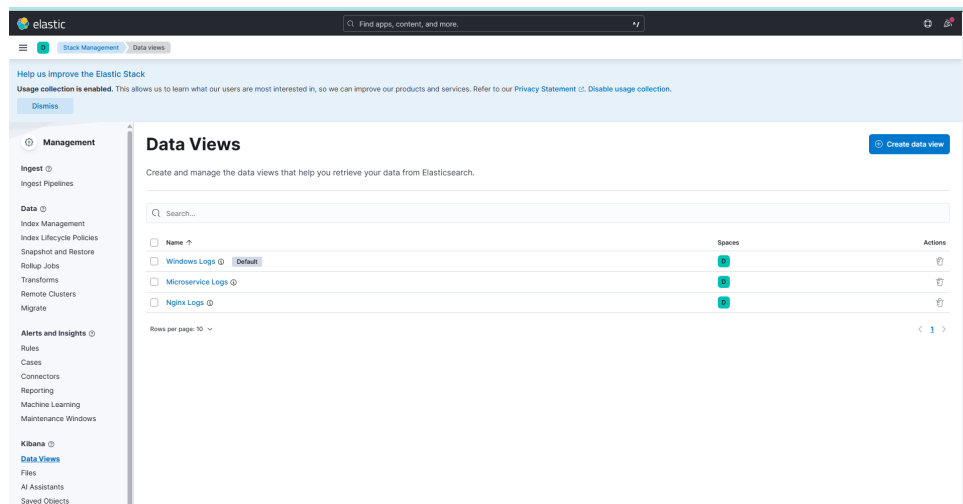


Figure 3: Kibana's Data views

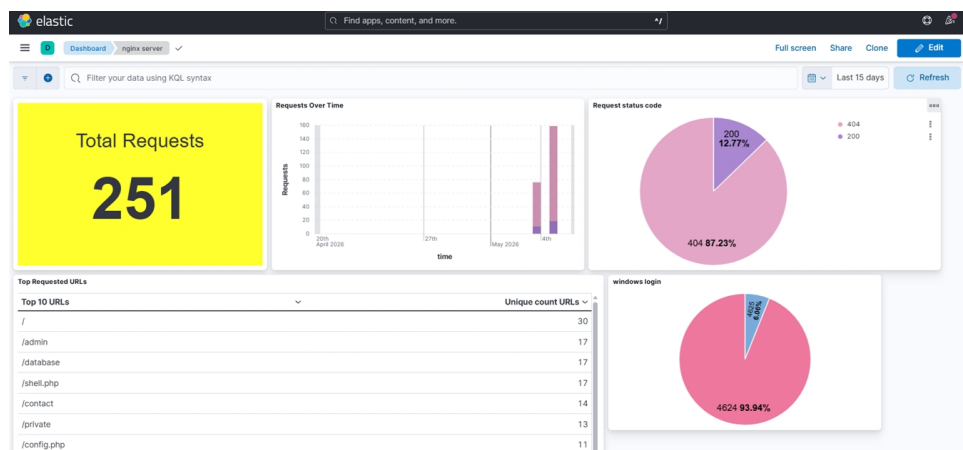


Figure 4: nginx dashboard

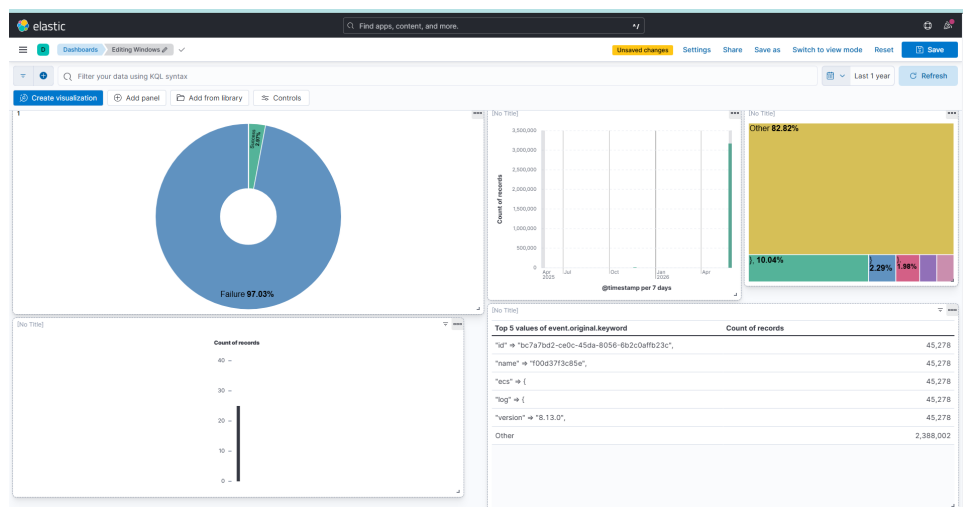


Figure 5: windows dashboard

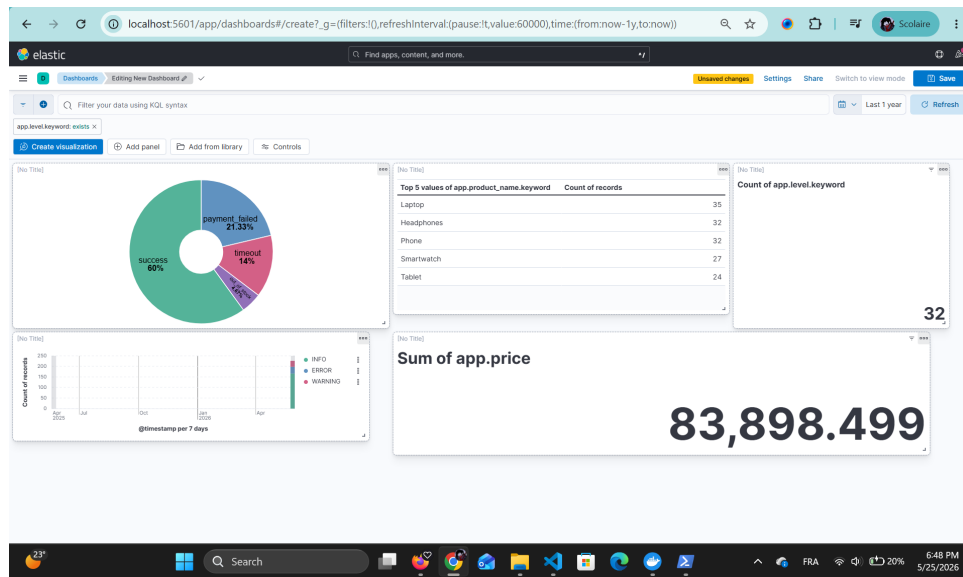


Figure 6: Microservice dashboard

7 Final Deliverable: System Interaction Map (Section 8)

7.1 Data Source 1: Ubuntu Linux Web Server (Nginx)

Source Name	Ubuntu Linux Web Server (Nginx)
Telemetry Type	Log
Shipper	Filebeat 8.7.1 — installed directly on the Ubuntu VM, monitors <code>/var/log/nginx/access.log</code>
Identity Protocol/Tool	TCP Beats Protocol over port 5044 — Filebeat opens an outbound TCP connection to Logstash on the host machine at <code>192.168.230.1:5044</code>
Processing / Transformation Step	Logstash applies a Grok filter using a combined regex pattern to parse each raw Nginx log line into structured fields: <code>client_ip</code> , <code>http_method</code> , <code>request_path</code> , <code>status_code</code> , <code>response_size</code> , <code>user_agent</code> , and <code>timestamp</code> . A Date filter normalises the timestamp into ISO-8601 <code>@timestamp</code> . A Mutate filter adds enrichment fields (<code>environment=lab</code> , <code>server=nginx</code>) and applies error or success tags based on HTTP status code.
Final Destination / Storage	Elasticsearch — daily rolling index <code>nginx-logs-YYYY.MM.dd</code> stored in the <code>esdata01</code> Docker volume
Visualization Tool	Kibana — Nginx Monitor Dashboard featuring: HTTP Status Distribution Pie Chart (200 vs 404), Requests Over Time bar chart, Top Requested URLs table, and Total Requests metric counter

7.2 Data Source 2: Windows Server Virtual Machine (Security Event Log)

Source Name	Windows Server Virtual Machine (Security Event Log)
Telemetry Type	Log
Shipper	Winlogbeat 8.7.1 — installed as a Windows Service on the Windows VM, subscribed to the Security event channel filtering for Event IDs 4624 and 4625
Identity Protocol/Tool	TCP Beats Protocol over port 5044 — Winlogbeat opens an outbound TCP connection to Logstash on the host machine at 192.168.230.1:5044
Processing / Transformation Step	Logstash receives pre-structured Windows Event Log JSON documents from Winlogbeat. A Mutate filter adds the enrichment field <code>service_type=windows</code> and <code>environment=lab</code> . Event ID 4624 represents a Successful Logon and Event ID 4625 represents a Failed Logon — both are routed to the Windows index without further parsing as Winlogbeat already structures the fields.
Final Destination / Storage	Elasticsearch — daily rolling index <code>windows-logs-YYYY.MM.dd</code> stored in the <code>esdata01</code> Docker volume
Visualization Tool	Kibana — Windows Security Dashboard featuring: Security Events table showing Event IDs, Failed vs Successful Logon comparison chart, and a brute-force detection timeline showing repeated Event ID 4625 spikes

7.3 Data Source 3: Checkout Microservice (Containerised Python/Flask Application)

Source Name	Checkout Microservice (Containerised Python/Flask Application)
Telemetry Type	Log
Shipper	Filebeat 8.7.1 running as a Docker container on the host machine, using the container input type. It reads all container log files from <code>/var/lib/docker/containers/*//*.log</code> via the Docker socket and uses the <code>add_docker_metadata</code> processor to attach container name, image, and ID. A <code>drop_event</code> processor filters out all containers except <code>checkout-microservice</code> .
Identity Protocol/Tool	TCP Beats Protocol over port 5044 — Filebeat communicates with Logstash using the internal Docker network name <code>logstash:5044</code> since both containers share the elastic Docker network
Processing / Transformation Step	Logstash detects the <code>container.name=checkout-microservice</code> field and applies a JSON filter to parse the structured application log into the <code>app</code> sub-object, extracting fields including <code>app.level</code> , <code>app.scenario</code> , <code>app.user_id</code> , <code>app.product_name</code> , <code>app.price</code> , and <code>app.latency_ms</code> . A Mutate filter adds <code>service_type=microservice</code> and tags events as error, warning, or success based on log level.
Final Destination / Storage	Elasticsearch — daily rolling index <code>microservice-logs-YYYY.MM.dd</code> stored in the <code>esdata01</code> Docker volume
Visualization Tool	Kibana — Checkout Microservice Monitor Dashboard featuring: Checkout Scenario Distribution Pie Chart (success / timeout / payment failed / out of stock), Checkout Events Timeline by log level, Total Revenue Sum metric, Most Requested Products table, and Total Errors and Warnings counter